

Ansible

[Sir's Favorite Tool]

classmate

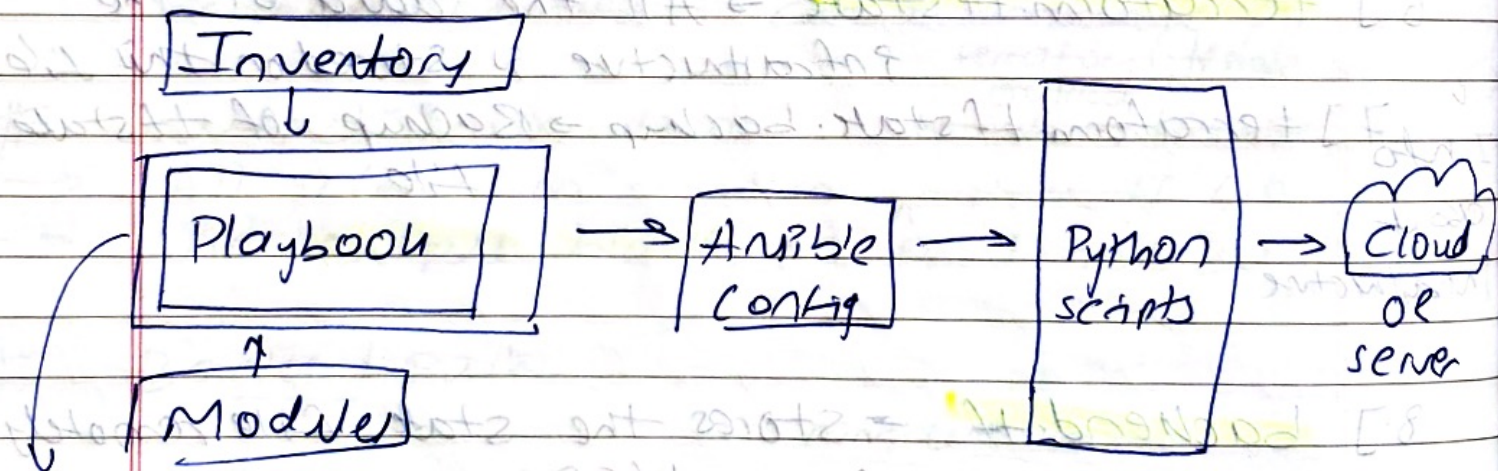
Date

Page 46

so much

- Used for, automation of
 - Configure automation
 - Application deployment automation
 - Infrastructure Provisioning automation along with Terraform.
 - CI/CD automation
 - Orchestration automation
 - Network automation → Integrate with so many tools in order to automate large scale automation
- YAML-based
- Agentless - connects over ssh.
- Scalable
- Simple & Easy to Read & Write
- It's just Python library

Ansible Architecture



called
Ansible
Playbook

"Simplicity is Beauty"

→ Principle of Ansible

[Use Notes in Intro To show

classmate

Date

Page 47

How much valuable this video is]

& perceive & show it as so much value

& premium CNO money can buy this]

[written for free]

Task-1:- Creating Inventory file:-

- Inventory file consist data of
 - targets
 - Its IP address, username, password, login key, port number of host
- For ssh purpose.
- Inventory file feed this data to Ansible in order to connect efficiently.

- INT OR YAML based

→ YAML - code [':' is used many times]

space all : Instance name. variables

```
2  --- hosts:
4  --- web01:
6  --- ansible-usr: ec2-user
    ansible-host: Private IP of host server
    ansible-ssh : client key.pem
    - private-key-file
```

--- web02: Name should be same as key pair name.

--- db01:

||

||

→ But what if we have 100's of server then we group them.

--- children:

--- webservers:

--- hosts:

--- web01:

--- web02:

} one group

--- dbservers:

--- hosts:

--- db01:

} 2nd group

[you can subgroup in a group too]

--- dc-oregon: } Parent group

--- children:

--- webservers:

--- dbservers:

} children subgroup

Ad-hoc command:

→ Commands that are one-liner

→ used to perform crucial yet rarely repeating task

→ So, there is no need to write entire playbook.

→ Generally for multiple servers.

Used can

- Ping All Host [Test connectivity]
- Run shell commands
- Copy File to Remote hosts
- Download File Using `get -url`
- Install a Package using `apt. / yum`
- Start a service
- Stop a service
- Restart a service
- Etc [we have many more]

Note: As Ansible is used for configuration management, suppose a package is already installed on a server, it will not again install it & changed state = false [In green]

- so very useful [Doesn't install again & again]

→ You can see adhoc command & use them from documentation or by `gpt`.

→ Configuration management tool are Idempotent i.e. If there is change in state then only a change/update is done.

Idempotent

Ansible Playbook:-

In bash scripting, the .sh file consist of series of commands,

Similarly - A playbook is a YAML based file that consist of different plays that defined a set of instructions for automating task.

Key concepts:-

Hosts: which machine to run task on

Task: Actual action to perform

[Eg:- Install a package, create a user]

Modules: Built-in ansible tools used in tasks

[Eg:- yum, apt, copy, file]

Variables: custom value

Handlers: Task triggered by notification

Name: Name / comment given. $\leftarrow$ - name $\square$ comment

State:

[- name]

& name is different

$\Rightarrow$ Indentation is Everything! $\hookrightarrow$ same to the

one change: host & task have same
ie no indentation.

→ Play-1

Eg: - host: webservgrp

tasks:

--- name: Install Apache

--- yum:

--- name: httpd

--- state: latest

--- name: Deploy config

--- copy:

--- src: file/httpd.conf

--- dest: /etc/httpd.conf

task-1

task-2

- host: dbsrvgrp

Level-1 tasks:

Level-2 [- name: Install PostgreSQL

--- yum:

Level-3 [- name: postgresql

--- state: latest

Play-2

→ IN YAML, name [hyphen [space] name]

↳ hyphen represents a 'list' in python.

- Every item in the list is a dictionary
[key & value] in YAML.

Modules:

- Now a lot of task, A LOT, can be done using Ansible for which we don't have to remember the code.
- Even SPr's say, as a devops engineer, you will have to visit ansible module documentation a lot to perform the task.
- Documentation have 100s of module which you can use to perform task on Ansible.
- Also - we have 'Ansible module Index' Just like a book Index to find module easily.
- As a devops engineer - we have a lot of task related to files - upload, update, configure, delete file from one server to other for which 'file module' helps a lot.
- Ansible executes task using python scripts & some plays requires dependency in order to complete the task.

Useful Modules:

1.] System & Package management -

- apt / yum / dnf / package
- pip - Manage python packages
- systemctl / systemd - Manage service
- reboot
- debug - to print output

2.] File Management

- copy
- backup
- stat - Get file or directory status
- file - set permission.
- template - Intelligent copy, read the file, get the desired data from format

3.] Cloud Infrastructure Automation

- ec2 / ec2-instance
- ec2-vol - For EBS management
- S3
- docker - container / docker-image
- K8s - Manage Kubernetes objects
- git
- cron

4.] User & Security Management

- User
- group
- authorized keys

Etc

if you can define variables & then use it to copy it to other smartly.
- Just change the variables & config.

Ansible configuration settings:

- Ansible have default configure settings, but if we want we can change it.

⇒ Order of Ansible config:-

- 1] ANSIBLE-CONFIG [environment variable if set]
- 2] ansible.cfg [in current directory]
- 3] ~/.ansible.cfg [in home directory [hidden]]
- 4] /etc/ansible/ansible.cfg [global config file]

↑
priority
↓
Repository
specified
&
most
used!

Variable:-

Variables in playbook:-

Defining: vars:

```
-- http_port: 80
-- sql_user: admin.
```

⇒ Inventory based [which can be used at global level]

Group
variable

```
group_vars/all
group_vars/groupname
```

```
host_vars/hostname
```


- You can define variable on a role-level too
- Fact variable: setup module
 - Automatically runned by ansible while executing the playbooks.

- store output: Register module
variable

Playbook → Return → Store
execution JSON output
in output variable
↓
which
can be used
by another task

How to use variables in playbooks:

name: " {{ dbname }}"

- Often, variables are defined outside, in a separate file for more reusability.

like in directory: group-vars/all

How to use variable from other files:

var: {{ var_output_name }}

var: {{ var_output_comment }}

→ Precedence of different variables are different, you can check it out on ansible documentation.

Provisioning server

→ What we will do & learn?

- NTP service on multi OS.
- Users & group
- Change configuration file
- Decision making.
- Loop
- Templates for configuration.
- Handled
- Ansible Roles.

Condition statement

- No if-else
- Use - 'when'
- Indentation = same as - name
: task + 2

→ task:

- name: "

"

when: ansible_distribution == "Ubuntu"

Loops

- name: Install ntp agent on ubuntu apt:

-- name: "{{item}}" ←

state: present

update-cache: yes

when: ansible_distribution == "Ubuntu"

loop:

-- - ntp

- wget

- git

- zip

- unzip

← Automatically
item-variable
gets its five
value one by
one

→ - Ansible - 'Simplicity is the key'

so no complex syntax of loops

→ Use documentation & GPT to write
loops & conditional statement.

Roles in Playbook:-

→ 'Roles' are used to simplify ^{& manageable} content in the playbook.

See a playbook consists of -

- Global Declaration

- Variables

- Tasks

- Files

- Templates

- Handler etc

→ Roles basically divide the file into particular directory, giving result in modularity & easy to manage shifts file to that folder.

- Used for reusability.

→ Also you search for ^{already} built role at Ansible Galaxy & then search the role.

→ Roles are generally used a high organization level.

→ # Mostly used for reusability.

##

Ansible ~~ss~~ handlers

→ A handler is a special kind of task that run only when notified by another task.

For task like - restarting a service
only if something changes.

→ One task 'notify' → 'handler' task.

tasks:

- name:

copy:

--- src: nginx.conf

dest: /etc/nginx/nginx.conf

notify - Restart nginx

handlers:

--- name: Restart nginx

service:

--- name: nginx

state: restarted.

notify only

if there is

change in state

→ Template module + handlers are used a lot.